

Quiz #6

CS 225 Data Structures

Spring 2026

7:00-8:00pm, Thursday, May 7

|        |              |
|--------|--------------|
| Name   | Hanbin Zheng |
| ZJU ID |              |
| Lab TA |              |

- This is a **closed book** and **closed notes** exam. No electronic aids are allowed, either.
- You should have 4 problems total on 7 pages. You must write on the exam.
- Unless otherwise stated in a problem, assume the best possible design of a particular implementation is being used.
- Unless the problem specifically says otherwise, (1) assume the code compiles, and thus any compiler error is an exam typo (though hopefully there are not any typos), and (2) assume you are NOT allowed to write any helper methods to help solve the problem, nor are you allowed to use additional arrays, lists, or other collection data structures unless we have said you can.
- Please put your name at the top of each page.
- Please fill in your **Lab Section** or your **Lab TA's name** to help us organize and grade your papers.
- Don't panic and good luck!

Problem 1 \_\_\_\_\_/10

Problem 2 \_\_\_\_\_/10

Problem 3 \_\_\_\_\_/10

Problem 4 \_\_\_\_\_/10

---

Total \_\_\_\_\_/40



Problem 1 (10 points): heaps

1. Consider the following min heap:

[0 1 2 3 4 5 6 7 8 9 10 11 12 13]  
[4, 9, 20, 15, 11, 30, 25, 28, 17, 19, 13]

Now we perform the following operations: first insert the value 6, then perform removeMin once.

a) After inserting 6, how many swaps are needed?

Answer: 2

b) After inserting 6, the value 6 is at index (the first element in the heap has index 1):

Answer: 3

c) After the removeMin, what is the final heap array?

- a) [6, 9, 7, 20, 15, 11, 30, 25, 28, 17, 19, 13]
- b) [6, 9, 7, 20, 15, 13, 30, 25, 28, 17, 19, 11]
- c) [7, 9, 11, 20, 15, 13, 30, 25, 28, 17, 19, 6]
- d) [6, 9, 11, 20, 15, 13, 30, 25, 28, 17, 19, 7]
- e) None of the above

2. Consider the unsorted array:

[18, 42, 7, 25, 3, 16, 11, 31, 9, 5, 28, 14]

Now we buildHeap in-place to build a min heap with this array. What is the content of the heap?

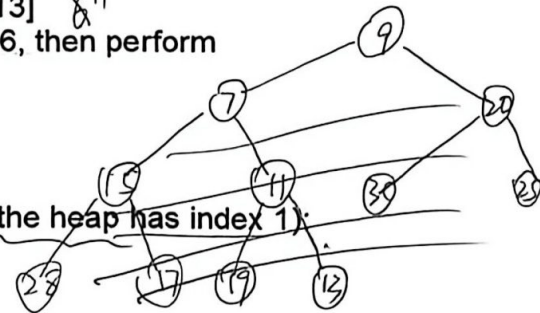
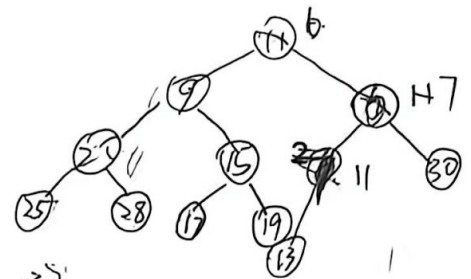
- a) [3, 5, 7, 9, 18, 14, 11, 31, 25, 42, 28, 16]
- b) [3, 5, 11, 9, 7, 14, 16, 42, 31, 25, 28, 18]
- c) [3, 5, 7, 9, 18, 16, 11, 31, 25, 42, 28, 14]
- d) [3, 5, 7, 9, 18, 14, 11, 25, 31, 42, 28, 16]
- e) None of the above

3. What is the worst-case running time of the algorithm to build a heap from an array containing n items that uses heapifyUp function?

- a)  $O(1)$
- b)  $O(\log n)$
- c)  $O(n)$
- d)  $O(n \log n)$
- e)  $O(n^2)$

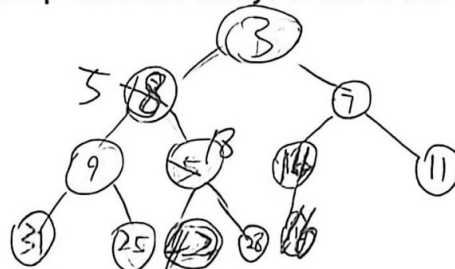
4. Consider a priority queue implemented using a binary heap (which could be a min-heap or a max-heap) containing n elements. What is the worst-case time complexity of the "pop" operation, which involves removing and returning the element with the highest priority?

- a)  $O(1)$
- b)  $O(\log n)$



6 9 7 20 15 11

31 25 42 28 16



- 1 1 1 1 1



- c)  $O(n)$
- d)  $O(n \log n)$
- e)  $O(n^2)$

5. What is the worst-case time complexity to find an arbitrary element (i.e., search for a specific value that might be anywhere in the heap, not necessarily the minimum or maximum) within a binary heap containing  $n$  elements?

- ~~a)  $O(1)$~~
- b)  $O(\log n)$
- ~~c)  $O(n)$~~
- d)  $O(n \log n)$
- ~~e)  $O(n^2)$~~



Problem2 Priority Queue ( Multiple choice, Select all correct choices)

C 1. A priority queue is implemented using an unsorted array containing  $n$  elements. What is the worst-case runtime of extractMin()?

- ☒ A.  $O(1)$
- ☒ B.  $O(\log n)$
- ☒ C.  $O(n)$
- ☒ D.  $O(n \log n)$

这题考的不多，这  $O(n)$  也是  $O(n \log n)$

A 2. Which of the following priority queue implementations can support insert() in worst-case  $O(1)$ ?

- ☒ A. Unsorted array
- ☒ B. Sorted array
- ☒ C. Unsorted linked list
- ☒ D. Binary min-heap

heap 对左右孩子

B C 3. In a binary min-heap, which statement is ALWAYS true?

BST

- ☒ A. The heap is also a binary search tree.
- ☒ B. Every parent node is less than or equal to its children.
- ☒ C. The largest element is always a leaf.
- ☒ D. An in-order traversal produces sorted order.

A D 4. Which operations can be performed in  $O(1)$  worst-case time using a sorted array implementation of a priority queue?

- ☒ A. peekMin()
- ☒ B. extractMin()
- ☒ C. insert(x)
- ☒ D. isEmpty()

5. Which application most naturally requires a priority queue ADT?

- ☒ A. Breadth-first search on an unweighted graph
- ☒ B. Reversing a linked list
- ☒ C. Dijkstra shortest-path algorithm
- ☒ D. Checking balanced parentheses

queue

graph is dijkstra 还没学，没办法了。





### Problem 3 (10 points): KDTree-Based Tile Matching in PhotoMosaic

In MP5, each tile image and each source-image region is represented by an average HSL color. To choose the best tile for a region, the color is converted into a 3-dimensional point and a KDTree<3> is used for nearest-neighbor search.

#### Given definitions and assumptions:

```
struct HSLAPixel { double h; double s; double l; }; // h in [0,360), s,l in [0,1]
struct TileInfo { HSLAPixel avgColor; };

template <int Dim> class Point {
public:
    Point(); Point(double x, double y, double z);
    double operator[](int index) const;
    bool operator<(const Point<Dim>& other) const;
};

template <int Dim> class KDTree {
public:
    KDTree(const vector<Point<Dim>>& points);
    Point<Dim> findNearestNeighbor(const Point<Dim>& query) const;
};
```

Assume all HSLAPixel values are valid. Point<3> can be used as a key in std::map; KDTree<3> is correctly implemented; findNearestNeighbor returns the closest point by squared Euclidean distance, and breaks distance ties using Point<3>::operator<.

#### A. HSL to Point conversion

Complete the function. Use H-S-L order and normalize hue to [0, 1].

```
Point<3> convertToPoint(const HSLAPixel& pixel) {
    return Point<3> (
        _____, // A1
        _____, // A2
        _____, // A3
    );
}
```

#### B. Complete the tile-matching code

The function returns result[i], the index of the closest tile to regions[i]. For duplicate tile colors, use the smallest tile index.

```
vector<int> matchTiles(const vector<HSLAPixel>& regions,
    const vector<TileInfo>& tiles) {
    if (regions.empty() || tiles.empty()) return {};
    vector<Point<3>> tilePoints;
    map<Point<3>, int> pointToIndex;

    for (unsigned i = 0; i < tiles.size(); i++) {
        Point<3> p = (tiles[i].avgColor); // B1
        tilePoints[i] = p; // B2
        if (pointToIndex[p] == -1) { // B3
            pointToIndex[p] = i; // B4
        }
    }

    KDTree<3> tree(tilePoints); // B5
    vector<int> result;
    for (unsigned i = 0; i < regions.size(); i++) {
        Point<3> query = convertToPoint(regions[i]); // B6
        Point<3> nearest = tree.findNearestNeighbor(query); // B7
        result.push_back(pointToIndex[nearest]); // B8
    }
}
```



```

    return result;
}

```

### C. Trace the code by hand

Suppose the tile list and source regions are as follows.

Tile list:

| Tile Index | HSL color       |
|------------|-----------------|
| 0          | (0, 1.0, 0.5)   |
| 1          | (180, 1.0, 0.5) |
| 2          | (90, 0.5, 0.5)  |
| 3          | (0, 1.0, 0.5)   |

Handwritten notes: (0, 1, 0.5), (0.5, 1, 0.5), (0.25, 0.5, 0.5), (0, 1, 0.5)

Source regions:

| Region Index | HSL color       |
|--------------|-----------------|
| 0            | (0, 1.0, 0.5)   |
| 1            | (180, 1.0, 0.5) |
| 2            | (90, 0.5, 0.5)  |
| 3            | (90, 0.75, 0.5) |

Handwritten notes: (0, 1, 0.5) → 0, (0.5, 1, 0.5) → 1, (0.25, 0.5, 0.5) → 2, (0.25, 0.75, 0.5) → 2

C1. Convert each tile color to Point<3>.

C2. What should matchTiles(regions, tiles) return?

Handwritten answer: [0, 1, 2, 2]

C3. Why should region 0 match tile 0 instead of tile 3? Answer in no more than 15 words.

### D. Find the bug

A student writes the following code when building the map from point to tile index:

```

for (unsigned i = 0; i < tiles.size(); i++) {
    Point<3> p = convertToPoint(tiles[i].avgColor);
    tilePoints.push_back(p);
    pointToIndex[p] = i;
}

```

What is the bug, and write one corrected line or block.



### Problem 4 (10 points): Disjoint sets and UpTree

Consider the following UpTree array:

| Index | 0 | 1 | 2 | 3 | 4  | 5 | 6 | 7  | 8 | 9 |
|-------|---|---|---|---|----|---|---|----|---|---|
| Arr   | 4 | 0 | 4 | 2 | -7 | 4 | 3 | -3 | 7 | 7 |

1. How many disjoint sets are stored in this structure? (1pt)

2. What is the answer of  $\text{find}(3)$ , according to the lecture? (1pt)

3. Perform path compression starting from node 3 based on the original array. Fill in the resulting array. (3pt)

| Index | 0 | 1 | 2 | 3 | 4  | 5 | 6 | 7  | 8 | 9 |
|-------|---|---|---|---|----|---|---|----|---|---|
| Arr   | 4 | 0 | 4 | 4 | -7 | 4 | 3 | -3 | 7 | 7 |

4. Now perform  $\text{union}(4, 7)$  by size based on the original array. Fill in the resulting array. (3pt)

| Index | 0 | 1 | 2 | 3 | 4  | 5 | 6 | 7 | 8 | 9 |
|-------|---|---|---|---|----|---|---|---|---|---|
| Arr   | 4 | 0 | 4 | 2 | -7 | 4 | 3 | 4 | 7 | 7 |

5. For an Disjoint Sets implemented with smart unions and path compression on find: Any sequence of  $m$  union and find operations result in the worse case running time of  $O(\underline{\hspace{2cm}})$ , where  $n$  is the number of items in the Disjoint Sets. (2pt)

